ADVANCED SQL
REAL-WORLD PRACTICE WORKSHEET

Joins • Subqueries • Window Functions • CASE • Mathematical & Aggregate Functions

Practice goal: Solve the problems yourself first. The worksheet deliberately does not provide solutions.

Scenario: You are a junior data analyst at *NovaCart*, an e-commerce company. Management wants customer, product, order and revenue insights from an operational SQL database.

SQL Dialect: MySQL 8+ / standard SQL-friendly syntax

Included: schema • sample data • question patterns • 75 practice questions • interview-style mixed challenges • answer-writing space

1. How to Use This Workbook

Recommended order: first understand the schema → write simple SELECTs → solve JOINs → subqueries → CASE/math → window functions → mixed problems.

Do not look for a single memorized query pattern. For each question, identify: **tables needed** → **relationship** → **filtering level** → **grouping level** → **calculation** → **final sorting/ranking**.

Important restriction: This workbook intentionally excludes CTEs, recursive queries, stored procedures, triggers, views, indexes, transactions, PIVOT, JSON, recursive SQL, and other advanced topics.

Core patterns you should master:

Topic	Patterns to practice
JOINs	INNER JOIN, LEFT JOIN, self-join thinking, multi-table joins, join + filter, join + GROUP BY
Subqueries	Scalar subquery, single-row comparison, IN, NOT IN, EXISTS, correlated subquery, subquery in FROM
Window functions	ROW_NUMBER, RANK, DENSE_RANK, LAG, LEAD, SUM OVER, AVG OVER, PARTITION BY, ORDER BY
CASE	Simple CASE, searched CASE, CASE in SELECT, CASE in ORDER BY, CASE inside aggregates
Math / Aggregates	SUM, AVG, MIN, MAX, COUNT, COUNT DISTINCT, ROUND, CEIL, FLOOR, ABS, percentage calculations
Mixed	Combining joins + subqueries + CASE + windows + math in realistic analytics tasks

Tip: For interview preparation, solve the same question in two ways when possible—for example, with a subquery and with a window function.

2. Database Schema — NovaCart

customers — one row per registered customer.

```
customers(
customer_id INT PRIMARY KEY,
customer_name VARCHAR(100),
city VARCHAR(50),
state VARCHAR(50),
signup_date DATE
)
```

products — product catalog and current inventory.

```
products(
product_id INT PRIMARY KEY,
product_name VARCHAR(100),
category VARCHAR(50),
unit_price DECIMAL(10,2),
stock_qty INT
)
```

orders — one row per order.

```
orders(
order_id INT PRIMARY KEY,
customer_id INT,
order_date DATE,
order_status VARCHAR(30),
payment_method VARCHAR(30),
total_amount DECIMAL(12,2)
)
```

order_items — line items inside each order.

```
order_items(
order_id INT,
product_id INT,
quantity INT,
line_total DECIMAL(12,2),
PRIMARY KEY(order_id, product_id)
)
```

Relationships:

Parent	Child	Relationship
customers.customer_id	orders.customer_id	One customer → many orders
orders.order_id	order_items.order_id	One order → many items
products.product_id	order_items.product_id	One product → many order items

Business rule for this workbook: **Cancelled** orders should generally be excluded when calculating completed sales/revenue unless a question explicitly says otherwise.

3. Populated Practice Data

3.1 customers

customer_id	customer_name	city	state	signup_date
101	Aarav Mehta	Nagpur	Maharashtra	2025-01-12
102	Diya Sharma	Pune	Maharashtra	2025-02-04
103	Kabir Patil	Mumbai	Maharashtra	2025-02-20
104	Anaya Joshi	Nashik	Maharashtra	2025-03-03
105	Rohan Deshmukh	Nagpur	Maharashtra	2025-03-18
106	Ishita Kulkarni	Aurangabad	Maharashtra	2025-04-02
107	Vivaan Rao	Bengaluru	Karnataka	2025-04-15
108	Meera Shah	Ahmedabad	Gujarat	2025-05-01
109	Arjun Nair	Kochi	Kerala	2025-05-19
110	Sara Khan	Hyderabad	Telangana	2025-06-07
111	Neha Verma	Delhi	Delhi	2025-06-15
112	Aditya Singh	Jaipur	Rajasthan	2025-06-20

3.2 products

product_id	product_name	category	unit_price	stock_qty
201	Laptop Pro 14	Electronics	65000	12
202	Wireless Mouse	Electronics	1200	60
203	Mechanical Keyboard	Electronics	3500	35
204	Office Chair	Furniture	8500	18
205	Standing Desk	Furniture	14000	10
206	USB-C Hub	Electronics	2200	40
207	Noise Cancelling Headphones	Electronics	9000	22
208	Webcam HD	Electronics	3000	27
209	Notebook Pack	Stationery	500	100
210	Desk Lamp	Furniture	1800	45
211	Monitor 24 Inch	Electronics	15000	16
212	Ergonomic Footrest	Furniture	2500	30
213	Bluetooth Speaker	Electronics	4500	25
214	Tablet Stand	Accessories	1800	40
215	Drawing Tablet	Electronics	12000	8

3.3 orders

order_id	customer_id	order_date	status	payment	total_amount
1001	101	2025-06-10	Delivered	UPI	65000
1002	102	2025-06-11	Delivered	Card	4700
1003	103	2025-06-12	Shipped	Card	17500
1004	101	2025-06-15	Delivered	Card	12400
1005	104	2025-06-18	Cancelled	UPI	8500
1006	105	2025-06-20	Delivered	Cash	14000
1007	106	2025-06-22	Delivered	UPI	2700
1008	107	2025-06-25	Shipped	Card	9000
1009	108	2025-06-28	Delivered	UPI	18100
1010	109	2025-07-01	Processing	Card	3500
1011	110	2025-07-03	Delivered	Card	10200
1012	103	2025-07-04	Delivered	UPI	16200
1013	105	2025-07-05	Cancelled	Card	1200
1014	107	2025-07-07	Delivered	UPI	17000
1015	102	2025-07-08	Delivered	Card	3000
1016	101	2025-07-10	Processing	UPI	4500
1017	110	2025-07-12	Delivered	Card	8000

3.4 order_items

order_id	product_id	quantity	line_total
1001	201	1	65000
1002	202	1	1200
1002	203	1	3500
1003	204	1	8500
1003	203	2	7000
1003	209	4	2000
1004	207	1	9000
1004	202	1	1200
1004	206	1	2200
1005	204	1	8500
1006	205	1	14000
1007	209	3	1500
1007	210	1	1200
1008	207	1	9000
1009	205	1	14000
1009	210	2	3600
1009	209	1	500
1010	203	1	3500
1011	207	1	9000
1011	202	1	1200
1012	205	1	14000
1012	206	1	2200
1013	202	1	1200
1014	205	1	14000
1014	210	1	1800
1014	202	1	1200
1015	208	1	3000
1016	213	1	4500
1017	213	1	4500
1017	203	1	3500

4. JOIN Practice — Fundamentals to Interview Level

Focus on choosing the correct join and controlling duplicate rows.

1. Display each order_id, customer_name, order_date and total_amount.

2. Display every customer and the orders they have placed. Include customers who have never placed an order.

3. Display order_id, customer_name, product_name and quantity for every order item.

4. Show product_name, category, quantity sold and line_total for all order items.

5. Find all orders placed by customers from Maharashtra.

6. Display customer_name, order_id and order_status for all orders that are not cancelled.

7. Find all products that have appeared in at least one order.

8. Find products that have never been ordered.

9. Show each customer with their total number of orders, including customers with zero orders.

10. Show each product with the total quantity sold. Include products with zero sales.

11. Find customers who have purchased a product from the Furniture category.

12. Find customers who have purchased both Electronics and Furniture products.

13. Find the top 5 customers by completed order value using joins and aggregation.

14. For each category, calculate total sales generated from its order items. Exclude cancelled orders.

15. Find the order with the highest number of distinct products.

16. For each order, show order_id, customer_name, number of distinct products, total quantity and total_amount.

17. Find customers who placed more than one order.

18. Find customers whose every order is cancelled or who have no completed order.

19. Show each state and the number of distinct customers, number of orders, and completed revenue.

20. Find the product that generated the highest line-item revenue among completed orders.

5. Subquery Practice — Scalar, IN, EXISTS, Correlated

Do not use CTEs. The point is to become comfortable nesting queries.

1. Find orders whose total_amount is greater than the average order amount across all orders.

2. Find customers whose total completed spending is greater than the average customer completed spending.

3. Find products priced above the average product price.

4. Find the most expensive product.

5. Find all customers who have placed at least one order using a subquery with IN.

6. Find customers who have never placed an order using NOT IN.

7. Find customers who have placed at least one completed order using EXISTS.

8. Find products that have never appeared in order_items using NOT EXISTS.

9. Find orders whose amount is greater than the average order amount for their own customer. Use a correlated subquery.

10. Find products whose price is greater than the average price of products in the same category.

11. Find customers whose latest order amount is greater than their own average order amount.

12. Find the second-highest product price without using LIMIT/OFFSET.

13. Find the third-highest distinct order amount.

14. Find customers who have placed more orders than the average number of orders per customer.

15. Find products whose total sold quantity is greater than the average sold quantity across products.

16. Find the customer(s) who placed the highest-value completed order.

17. Find orders that contain the product with the highest unit price.

18. Find customers who purchased the most expensive product.

19. Find categories whose total completed revenue is above the average category revenue.

20. Find the highest-spending customer in each state using a correlated subquery or nested aggregation.

6. CASE Statement Practice — Business Rules

Practice searched CASE first. Then try CASE inside aggregates and ORDER BY.

1. Display each product with a stock_status: 'Out of Stock', 'Low Stock' for stock_qty < 15, 'Medium Stock' for 15–30, otherwise 'Healthy Stock'.

2. Classify each order as 'High Value' if total_amount >= 15000, 'Medium Value' if >= 5000, otherwise 'Low Value'.

3. Create a payment_type label: UPI, Card, Cash, or Other.

4. Display customer_name and a customer_region label: Maharashtra vs Other State.

5. For each order, create a revenue_status: Completed, Cancelled, or Pending based on order_status.

6. Calculate an estimated discount: 10% for orders >= 15000, 5% for orders >= 5000, otherwise 0%.

7. Calculate the final_amount after the estimated discount.

8. Count how many orders fall into each order-value band.

9. For each customer, classify them as 'New', 'Regular', or 'VIP' based on completed spending: <5000, 5000–14999, >=15000.

10. Use CASE inside SUM to calculate completed revenue and cancelled revenue separately in one query.

11. Use CASE inside COUNT to count completed orders and cancelled orders per customer.

12. Sort products so Electronics appear first, then Furniture, then Stationery, without changing the stored category values.

13. Create a delivery_priority label: High for high-value non-cancelled orders, Medium for other non-cancelled orders, Low for cancelled orders.

14. For each state, calculate the number of customers with at least one completed order using CASE.

15. Calculate a stock_risk score: 3 when stock < 10, 2 when stock is 10–19, 1 when stock is 20–30, 0 otherwise.

7. Mathematical & Aggregate Function Practice

Use SUM, AVG, COUNT, COUNT DISTINCT, MIN, MAX and numeric functions such as ROUND, CEIL, FLOOR and ABS.

1. Find total gross order value across all orders.

2. Find total completed revenue excluding cancelled orders.

3. Find average order value.

4. Find minimum and maximum order amount.

5. Find the number of unique customers who have placed orders.

6. Find the number of orders, total quantity and total revenue for each product.

7. Calculate the average selling price per product from order_items using line_total / quantity.

8. Calculate each product's revenue contribution percentage of total completed revenue.

9. Calculate each category's revenue contribution percentage of total completed revenue.

10. Round each customer's average completed order value to 2 decimal places.

11. Find the absolute difference between a product's unit_price and its average category price.

12. Calculate the value of current stock for every product as unit_price * stock_qty.

13. Find the total inventory value across all products.

14. Calculate an order's average item value as total_amount divided by total quantity in that order.

15. Find the average number of items per completed order.

16. Find products where current inventory value is below 30000.

17. Use CEIL to calculate how many full packs of 10 units would be required to cover each product's current stock quantity.

18. Use FLOOR to calculate the number of complete ■5,000 sales blocks in each completed order.

19. Calculate the percentage of all orders that are cancelled.

20. Calculate the percentage of customers who have never placed an order.

8. Window Function Practice — Ranking & Comparisons

Use window functions only. Avoid CTEs. When a window result must be filtered, use a subquery in FROM.

1. Assign ROW_NUMBER to orders ordered by total_amount descending.

2. Rank all products by unit_price using RANK.

3. Rank products within each category by unit_price using RANK and PARTITION BY.

4. Compare RANK and DENSE_RANK on products by price. Identify where the results would differ if ties existed.

5. Assign a row number to each customer's orders, ordered from oldest to newest.

6. Find each customer's first order using ROW_NUMBER in a subquery.

7. Find each customer's latest order using ROW_NUMBER in a subquery.

8. Calculate a running total of completed revenue ordered by order_date.

9. Calculate a running total of completed revenue separately for each customer.

10. Calculate each order's percentage of the total completed revenue using SUM() OVER().

11. Calculate each customer's percentage contribution to total completed revenue using a window function.

12. Calculate the average order value for each customer alongside every order using AVG() OVER(PARTITION BY ...).

13. Show each order amount alongside the previous order amount for the same customer using LAG.

14. Show each order amount alongside the next order amount for the same customer using LEAD.

15. Calculate the difference between the current order amount and the previous order amount for each customer.

16. Find the highest-value order for each customer using RANK or ROW_NUMBER.

17. Find the top 2 products by price within each category.

18. Find the top 2 customers by completed spending within each state.

19. For each product, show total quantity sold and its rank by quantity within its category.

20. Calculate the cumulative quantity sold for each product category ordered by product price descending.

9. Mixed Real-World Analyst Challenges

These are intentionally less direct. Decide which techniques to combine before writing SQL.

1. Management wants the top 3 customers by completed spending in each state. Return state, customer_name, spending and rank.

2. Find the first completed order for every customer and show how much that order was worth.

3. Find customers whose latest completed order is larger than their average completed order.

4. For every category, return the highest-revenue product and its revenue.

5. Find the product that contributes the largest percentage of revenue within its own category.

6. Identify customers whose completed spending is above the average completed spending of customers in their state.

7. For each customer, show completed spending, customer rank within state, and percentage contribution to their state's completed revenue.

8. Find orders that are above the overall average order amount but below their customer's average order amount.

9. Find the month-independent sequence of each customer's orders and calculate the change in amount from one order to the next.

10. For each category, show total quantity sold, total revenue, average product price, and the highest-priced product.

11. Find customers who bought at least one product whose current stock is below 15.

12. Find customers who bought from every product category represented in the data.

13. Find the state with the highest completed revenue and return its revenue plus percentage of company revenue.

14. Calculate completed revenue, cancelled revenue, pending revenue, and cancellation percentage for each payment method.

15. Find products whose total completed sales quantity is above the average quantity sold across all products.

16. For each customer, classify their spending using CASE and also rank them by spending within their state.

17. Find the order containing the largest total quantity of items and display customer_name and product names.

18. Calculate each order's discount using CASE, then show the adjusted amount and running adjusted revenue by date.

19. Find the second-highest-spending customer in each state.

20. Create a management report with customer_name, state, total orders, completed spending, average order value, latest order date, customer segment, and rank within state.

10. Pattern Drills — Repeat Until Automatic

These are short drills. Try writing each from memory without checking previous questions.

1. JOIN + WHERE: Get all completed orders for Maharashtra customers.

2. JOIN + GROUP BY: Get revenue by customer.

3. LEFT JOIN + COUNT: Get order count for every customer including zero.

4. LEFT JOIN + COALESCE: Display zero instead of NULL for customers with no orders.

5. JOIN + GROUP BY + HAVING: Find customers with completed spending above ■15,000.

6. Scalar subquery: Find rows greater than the overall average.

7. IN subquery: Find customers connected to orders matching a condition.

8. EXISTS: Test whether a related record exists.

9. Correlated subquery: Compare a row with its own group's average.

10. Subquery in FROM: Create an aggregated result first, then filter or rank it.

11. CASE in SELECT: Turn numeric values into business labels.

12. CASE in SUM: Conditional aggregation.

13. CASE in ORDER BY: Custom business sorting.

14. ROW_NUMBER: Pick the first or latest row per group.

15. RANK: Find top N within each group.

16. LAG: Compare current row with previous row.

17. LEAD: Compare current row with next row.

18. SUM OVER: Running total.

19. SUM OVER without PARTITION BY: Percentage of grand total.

20. SUM OVER with PARTITION BY: Percentage of group total.

21. ROUND: Format calculated business metrics.

22. ABS: Measure absolute difference.

23. CEIL/FLOOR: Convert a continuous calculation into whole business units.

11. Mini SQL Interview — No Hints

Set a timer for 45–60 minutes. Do not refer to the earlier questions while solving.

1. Return the top 2 customers by completed spending in every state.

2. Return the latest completed order for every customer, including customers who have no completed order.

3. Return products priced above their category's average price.

4. Return each customer's completed spending, average completed order amount, spending rank within state, and a CASE-based customer segment.

5. Return each category's revenue, percentage of company revenue, top product by revenue, and total quantity sold.

6. Find orders whose value is above the overall average but below the customer's own average.

7. Return the second-highest distinct product price and all products having that price.

8. Find customers who have purchased from both Electronics and Furniture.

9. For each order, return previous order amount for that customer and the difference from the previous order.

10. Build a final management query showing state, customer, completed spending, order count, average order value, latest order date, rank, and segment.

12. Self-Check Checklist

Skill	Can I do this without help?	Score / 5
INNER JOIN + multiple tables	■	
LEFT JOIN and handling missing rows	■	
JOIN + GROUP BY + HAVING	■	
Scalar subquery	■	
IN / NOT IN subquery	■	
EXISTS / NOT EXISTS	■	
Correlated subquery	■	
Subquery in FROM	■	
CASE in SELECT	■	
CASE inside SUM/COUNT	■	
CASE in ORDER BY	■	
ROW_NUMBER	■	
RANK / DENSE_RANK	■	
LAG / LEAD	■	
Running totals with SUM OVER	■	
PARTITION BY	■	
Percentage calculations with windows	■	
Math + aggregate functions	■	
Combining multiple techniques	■	

Recommended practice rule: **Do not just get the output—explain why your query works.** In interviews, be ready to explain join choice, duplicate-row risk, grouping level, NULL behavior, and why a window function or subquery is appropriate.

End of worksheet — keep this PDF as your question bank and create a separate SQL file for your solutions.